

```

# Data Manipulation
import pandas as pd
import numpy as np

# Visualization
import matplotlib.pyplot as plt
import seaborn as sns

# Ignore Warnings
import warnings
warnings.filterwarnings('ignore')

# Display graphs inside notebook
%matplotlib inline

# Read Excel Dataset
df = pd.read_excel("/content/sample_data/flood dataset.xlsx")

# Display first five rows
df.head()

print("Rows :", df.shape[0])
print("Columns :", df.shape[1])

Rows : 115
Columns : 11

df.columns

Index(['Temp', 'Humidity', 'Cloud Cover', 'ANNUAL', 'Jan-Feb', 'Mar-
May',
      'Jun-Sep', 'Oct-Dec', 'avgjune', 'sub', 'flood'],
      dtype='object')

df.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 115 entries, 0 to 114
Data columns (total 11 columns):
#   Column          Non-Null Count  Dtype
---  -
0   Temp            115 non-null    int64
1   Humidity        115 non-null    int64
2   Cloud Cover     115 non-null    int64
3   ANNUAL          115 non-null    float64
4   Jan-Feb         115 non-null    float64
5   Mar-May         115 non-null    float64
6   Jun-Sep         115 non-null    float64
7   Oct-Dec         115 non-null    float64
8   avgjune         115 non-null    float64
9   sub             115 non-null    float64

```

```
10 flood          115 non-null    int64
dtypes: float64(7), int64(4)
memory usage: 10.0 KB
```

```
df.dtypes
```

```
Temp          int64
Humidity       int64
Cloud Cover   int64
ANNUAL        float64
Jan-Feb       float64
Mar-May       float64
Jun-Sep       float64
Oct-Dec       float64
avgjune       float64
sub           float64
flood         int64
dtype: object
```

```
df.isnull().sum()
```

```
Temp          0
Humidity       0
Cloud Cover    0
ANNUAL         0
Jan-Feb        0
Mar-May        0
Jun-Sep        0
Oct-Dec        0
avgjune        0
sub            0
flood          0
dtype: int64
```

```
df.describe()
```

```
{"summary":{"name": "df", "rows": 8, "fields": [{"column": "Temp", "properties": {"dtype": "number", "std": 33.14850216404906, "min": 1.1223409120258498, "max": 115.0, "num_unique_values": 7, "samples": [115.0, 29.6, 30.0], "semantic_type": ""}, {"column": "Humidity", "properties": {"dtype": "number", "std": 30.8671243976302, "min": 2.947623122916514, "max": 115.0, "num_unique_values": 8, "samples": [73.85217391304347, 74.0, 115.0], "semantic_type": ""}, {"column": "Cloud Cover", "properties": {"dtype": "number", "std": 33.14850216404906, "min": 1.1223409120258498, "max": 115.0, "num_unique_values": 7, "samples": [115.0, 29.6, 30.0], "semantic_type": ""}]}}
```

```

31.75429240283141,\n          \"min\": 4.330157846057955,\n
\"max\": 115.0,\n          \"num_unique_values\": 8,\n
\"samples\": [\n          36.28695652173913,\n          36.0,\n
115.0\n          ],\n          \"semantic_type\": \"\",\n
\"description\": \"\"\n          }\n          },\n          {\n          \"column\":
\"ANNUAL\",\n          \"properties\": {\n          \"dtype\": \"number\",\n
\"std\": 1405.9663262680265,\n          \"min\": 115.0,\n
\"max\": 4257.8,\n          \"num_unique_values\": 8,\n
\"samples\": [\n          2925.487826086957,\n          2937.5,\n
115.0\n          ],\n          \"semantic_type\": \"\",\n
\"description\": \"\"\n          }\n          },\n          {\n          \"column\":
\"Jan-Feb\",\n          \"properties\": {\n          \"dtype\": \"number\",\n
\"std\": 41.87503718123825,\n          \"min\": 0.3,\n
\"max\": 115.0,\n          \"num_unique_values\": 8,\n
\"samples\": [\n          27.739130434782613,\n          20.5,\n
115.0\n          ],\n          \"semantic_type\": \"\",\n
\"description\": \"\"\n          }\n          },\n          {\n          \"column\":
\"Mar-May\",\n          \"properties\": {\n          \"dtype\": \"number\",\n
\"std\": 266.0720239184206,\n          \"min\": 89.9,\n
\"max\": 915.2,\n          \"num_unique_values\": 8,\n
\"samples\": [\n          377.2539130434783,\n          342.0,\n
115.0\n          ],\n          \"semantic_type\": \"\",\n
\"description\": \"\"\n          }\n          },\n          {\n          \"column\":
\"Jun-Sep\",\n          \"properties\": {\n          \"dtype\": \"number\",\n
\"std\": 1074.8050769430715,\n          \"min\": 115.0,\n
\"max\": 3451.3,\n          \"num_unique_values\": 8,\n
\"samples\": [\n          2022.8408695652174,\n          1948.7,\n
115.0\n          ],\n          \"semantic_type\": \"\",\n
\"description\": \"\"\n          }\n          },\n          {\n          \"column\":
\"Oct-Dec\",\n          \"properties\": {\n          \"dtype\": \"number\",\n
\"std\": 251.20402487469605,\n          \"min\": 115.0,\n
\"max\": 823.3,\n          \"num_unique_values\": 8,\n
\"samples\": [\n          497.6365217391305,\n          501.5,\n
115.0\n          ],\n          \"semantic_type\": \"\",\n
\"description\": \"\"\n          }\n          },\n          {\n          \"column\":
\"avgjune\",\n          \"properties\": {\n          \"dtype\": \"number\",\n
\"std\": 103.4768452744339,\n          \"min\":
62.54759707472398,\n          \"max\": 366.0666666666667,\n
\"num_unique_values\": 8,\n          \"samples\": [\n
218.10086956521738,\n          211.03333333333333,\n          115.0\n
],\n          \"semantic_type\": \"\",\n          \"description\": \"\"\n
}\n          },\n          {\n          \"column\": \"sub\",\n          \"properties\": {\n
\"dtype\": \"number\",\n          \"std\": 300.94387682648414,\n
\"min\": 34.19999999999999,\n          \"max\": 982.7,\n
\"num_unique_values\": 8,\n          \"samples\": [\n
439.8017391304349,\n          430.6,\n          115.0\n
],\n          \"semantic_type\": \"\",\n          \"description\": \"\"\n
}\n          },\n          {\n          \"column\": \"flood\",\n          \"properties\": {\n
\"dtype\": \"number\",\n          \"std\": 40.58499981047687,\n

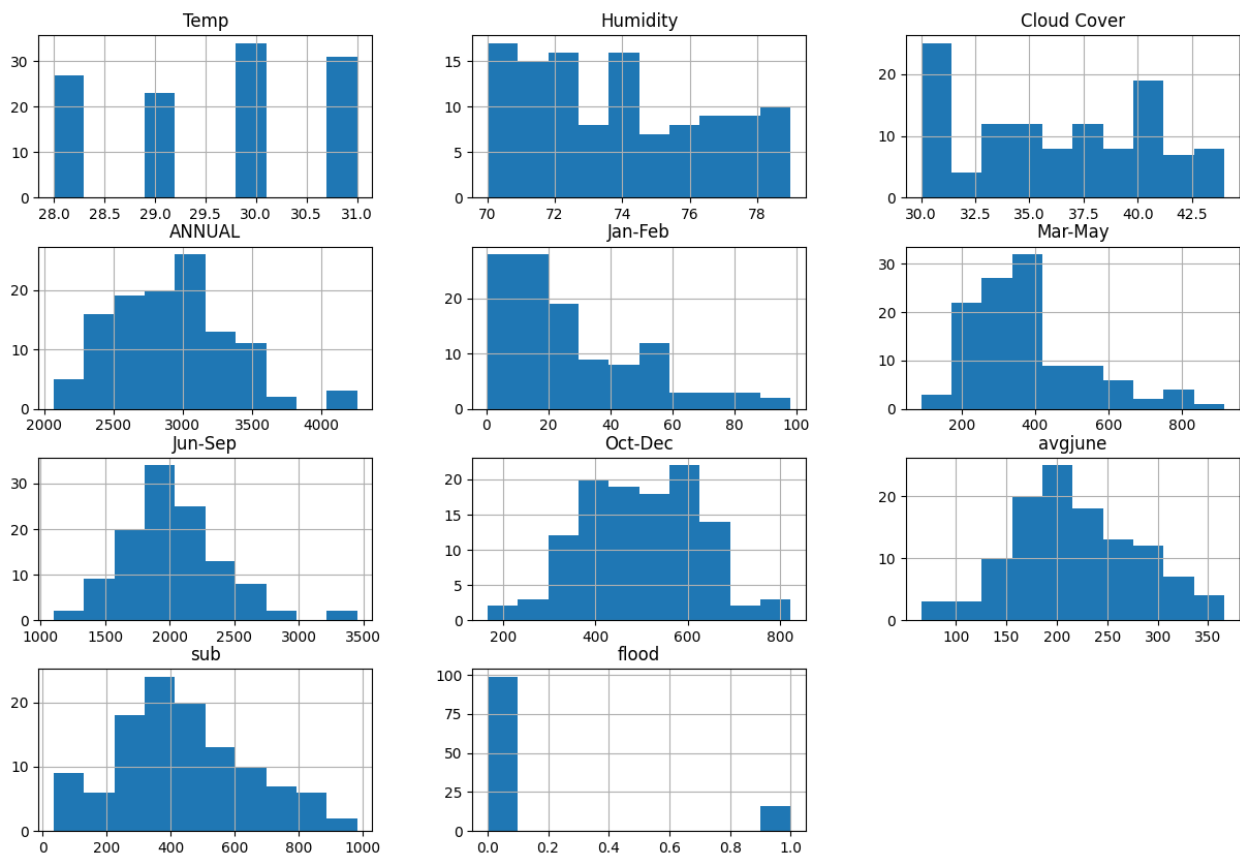
```

```
\ "min\": 0.0,\n          \ "max\": 115.0,\n          \ "num_unique_values\":  
5,\n          \ "samples\": [\n          0.1391304347826087,\n          1.0,\n          0.347597178681016\n          ],\n          \ "semantic_type\": \ "\",\n          \ "description\": \ "\">\n          }\n          ]\n          }", "type": "dataframe"}
```

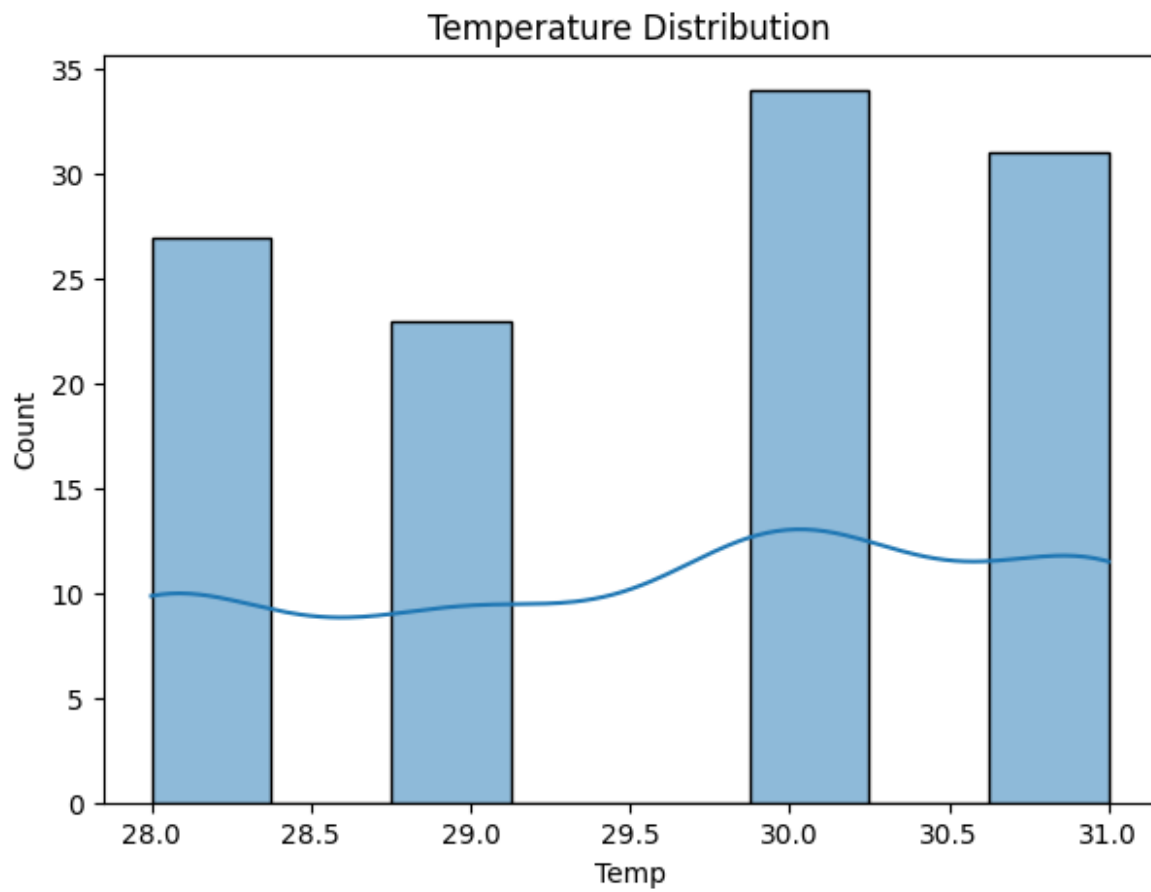
```
df['flood'].value_counts()
```

```
flood  
0    99  
1    16  
Name: count, dtype: int64
```

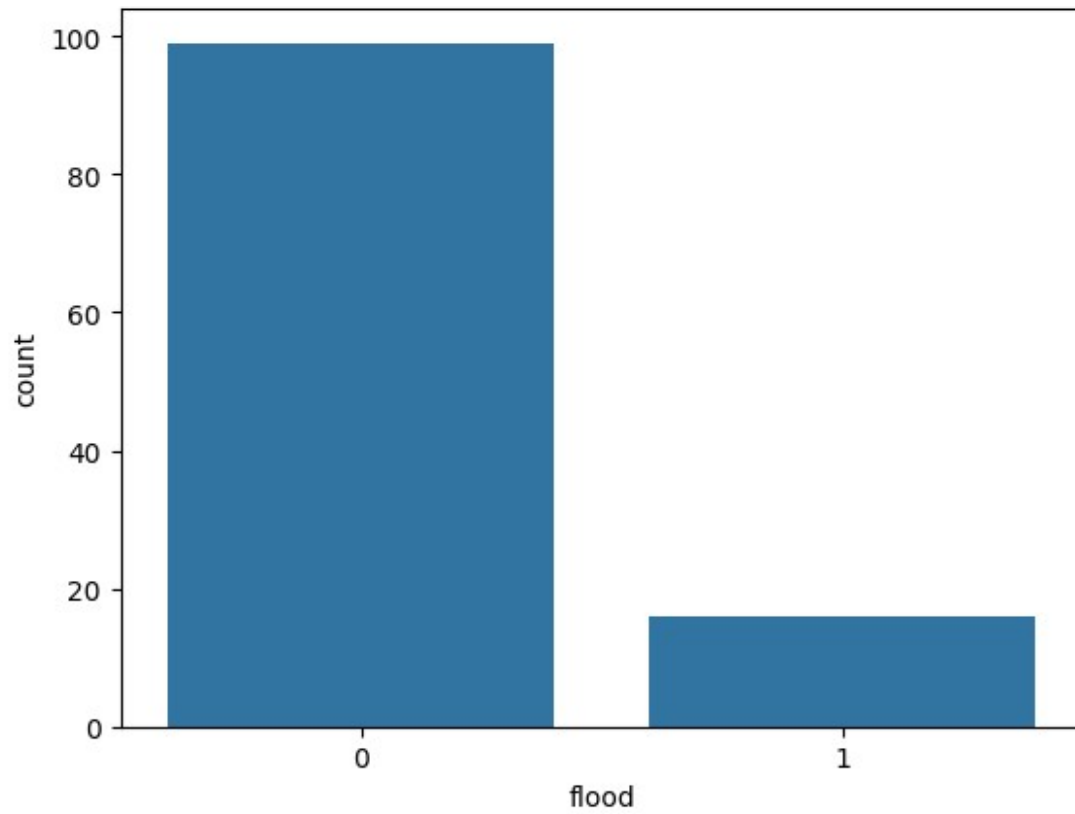
```
#Histogram  
df.hist(figsize=(15,10))  
plt.show()
```



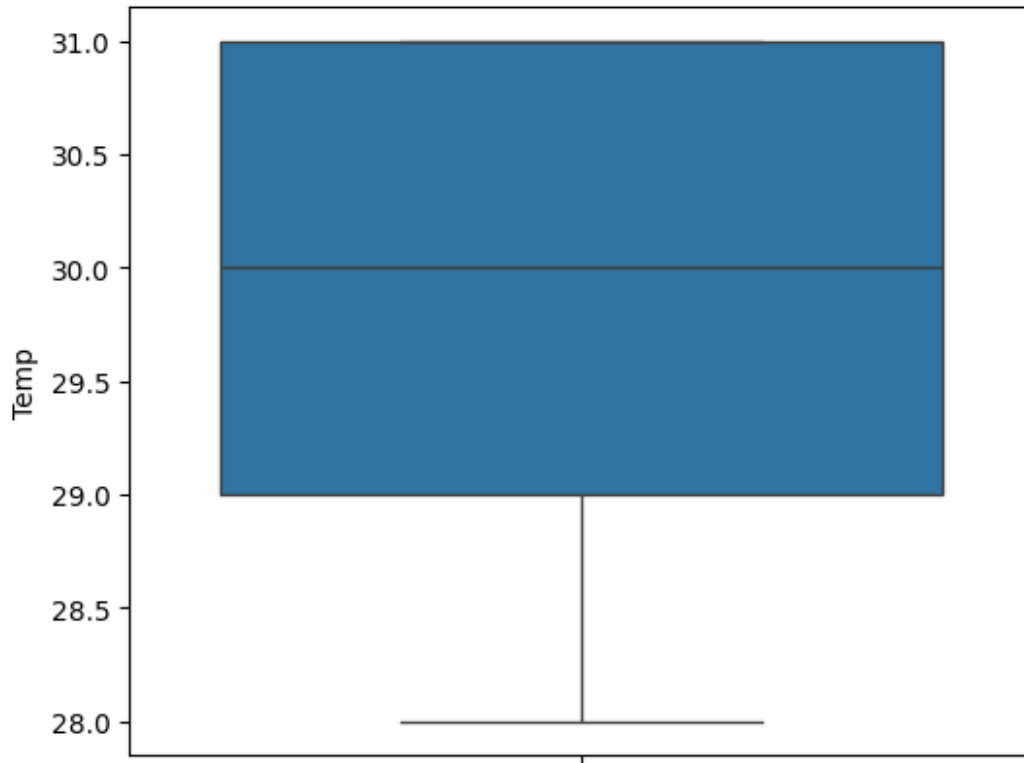
```
#Distribution plot  
plt.figure(figsize=(7,5))  
sns.histplot(df['Temp'], kde=True)  
plt.title("Temperature Distribution")  
plt.show()
```



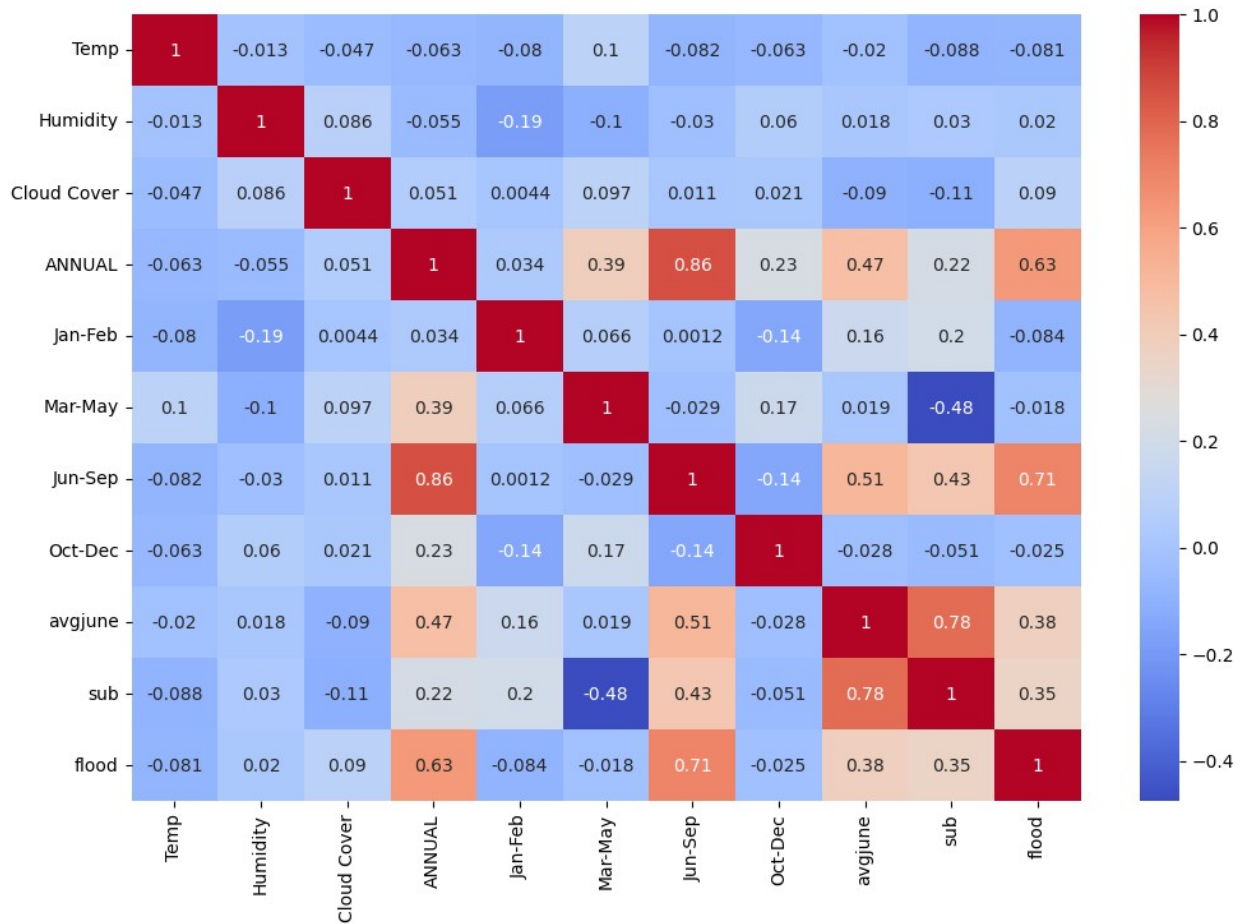
```
#Count plot  
sns.countplot(x='flood', data=df)  
plt.show()
```



```
#Box plot  
plt.figure(figsize=(6,5))  
sns.boxplot(y=df['Temp'])  
plt.show()
```



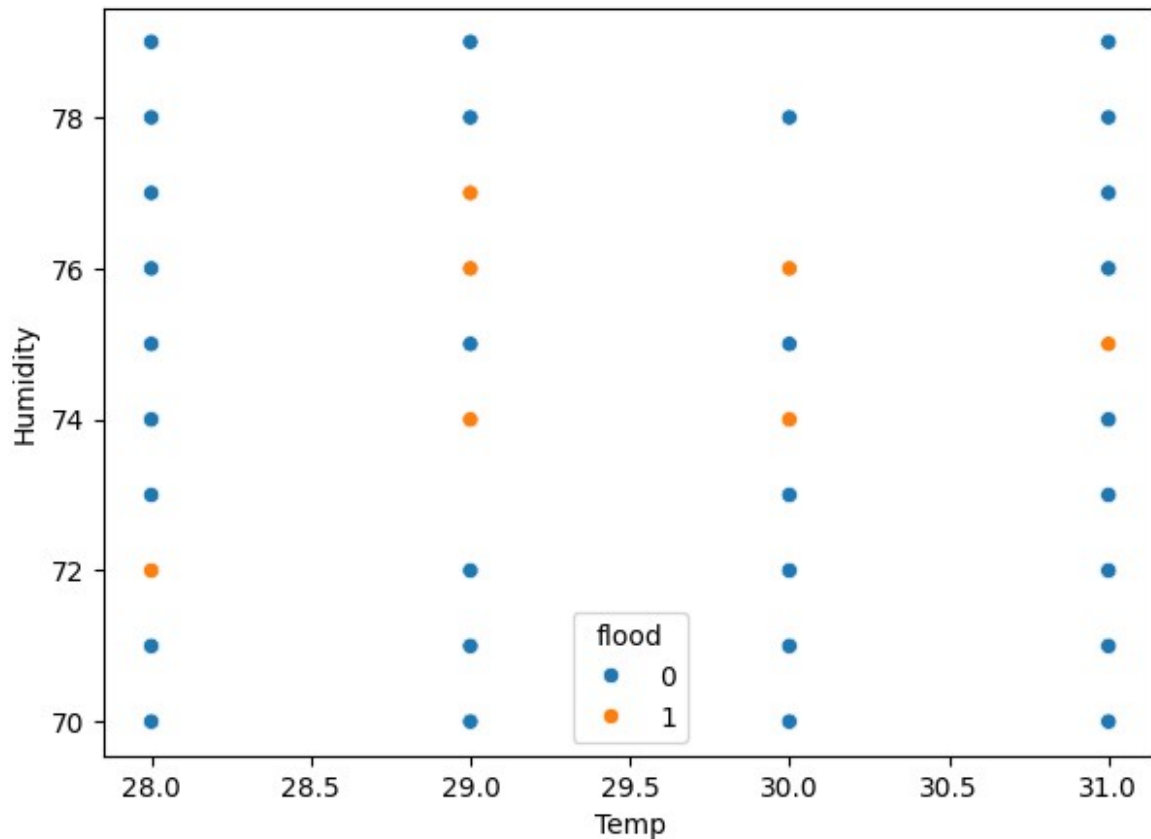
```
#Correlation matrix  
corr = df.corr()  
plt.figure(figsize=(12,8))  
sns.heatmap(corr,  
            annot=True,  
            cmap='coolwarm')  
plt.show()
```



```
#Scatter plot
plt.figure(figsize=(7,5))

sns.scatterplot(
    x='Temp',
    y='Humidity',
    hue='flood',
    data=df
)

plt.show()
```

#Mean, Median, Standard deviation, Variation, Correlation

```
df.mean(numeric_only=True)
```

```
Temp          29.600000
Humidity       73.852174
Cloud Cover    36.286957
ANNUAL        2925.487826
Jan-Feb       27.739130
Mar-May       377.253913
Jun-Sep       2022.840870
Oct-Dec       497.636522
avgjune       218.100870
sub           439.801739
flood          0.139130
dtype: float64
```

```
df.median(numeric_only=True)
```

```
Temp          30.000000
Humidity       74.000000
Cloud Cover    36.000000
ANNUAL        2937.500000
Jan-Feb       20.500000
Mar-May       342.000000
```

```
Jun-Sep      1948.700000
Oct-Dec      501.500000
avgjune      211.033333
sub          430.600000
flood        0.000000
dtype: float64
```

```
df.std(numeric_only=True)
```

```
Temp          1.122341
Humidity       2.947623
Cloud Cover    4.330158
ANNUAL        422.112193
Jan-Feb        22.361032
Mar-May       151.091850
Jun-Sep       386.254397
Oct-Dec       129.860643
avgjune        62.547597
sub           210.438813
flood          0.347597
dtype: float64
```

```
df.var(numeric_only=True)
```

```
Temp          1.259649
Humidity       8.688482
Cloud Cover   18.750267
ANNUAL       178178.703535
Jan-Feb       500.015736
Mar-May      22828.747068
Jun-Sep     149192.459105
Oct-Dec     16863.786549
avgjune      3912.201900
sub         44284.493857
flood         0.120824
dtype: float64
```

```
df.duplicated().sum()
```

```
np.int64(0)
```

```
# Check missing values
```

```
df.isnull().sum()
```

```
Temp          0
Humidity       0
Cloud Cover    0
ANNUAL         0
Jan-Feb        0
Mar-May        0
Jun-Sep        0
```

```
Oct-Dec      0
avgjune      0
sub          0
flood        0
```

```
dtype: int64
```

```
#Feature selection
```

```
X = df.drop("flood", axis=1)
```

```
y = df["flood"]
```

```
#Train test split
```

```
from sklearn.model_selection import train_test_split
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y,  
test_size=0.2, random_state=42)
```

```
print("Training Data :", X_train.shape)
```

```
print("Testing Data :", X_test.shape)
```

```
Training Data : (92, 10)
```

```
Testing Data : (23, 10)
```

```
#StandardScaler
```

```
from sklearn.preprocessing import StandardScaler
```

```
scaler = StandardScaler()
```

```
X_train = scaler.fit_transform(X_train)
```

```
X_test = scaler.transform(X_test)
```

```
print(X_train[:5])
```

```
[[-0.61039689 -0.66344859  0.15886699  1.08604492 -0.35983282  
 2.57623804  
 -0.53467443  2.2628558  -1.6046566  -0.59664114]  
 [-0.61039689  0.67069939 -0.8320322  1.06253895  0.45537729 -  
 0.39714984  
  1.68429033 -1.35274855  0.32161235  0.87878852]  
 [ 1.23080028  0.67069939 -0.3365826  1.10384556  1.0579239 -  
 0.38839723  
  1.15383728  0.36324965  1.12702475  1.35588824]  
 [ 0.3102017  1.33777339 -0.3365826  0.46713037 -0.35097184 -  
 0.09455949  
 -0.21844281  2.46118335  0.16497428  0.24379968]  
 [-1.53099547 -1.33052258 -1.57520658  1.27249521 -0.09843392 -  
 1.08610557  
  2.21134304 -1.37134176  0.46307442  1.05580085]]
```

```
import joblib
```

```
joblib.dump(scaler, "scaler.pkl")
```

```
print("Scaler saved successfully!")
```

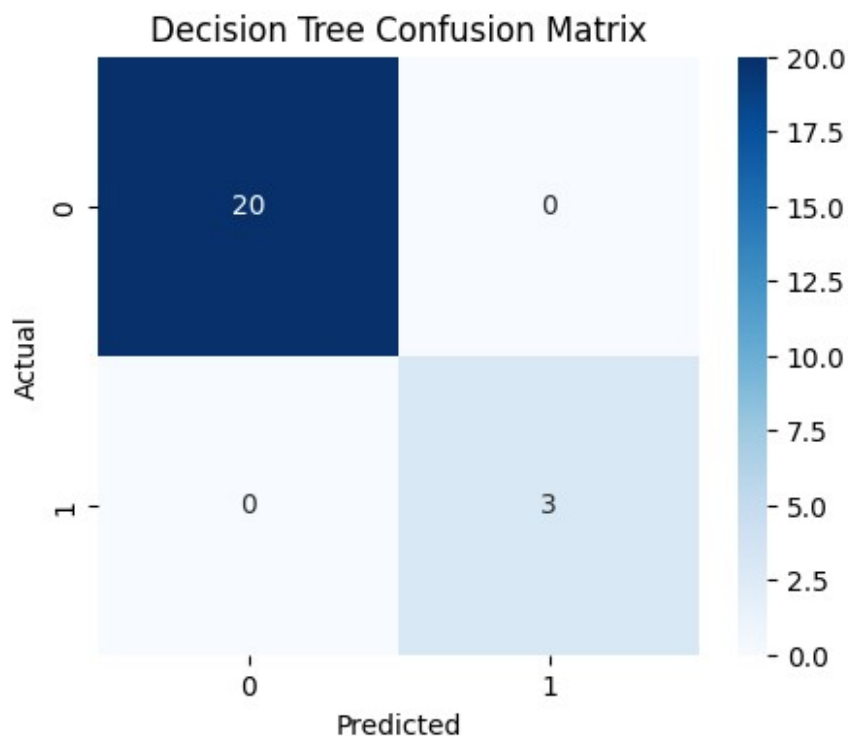
```
Scaler saved successfully!
```

```
print("Training Features :", X_train.shape)
print("Testing Features :", X_test.shape)
print("Training Labels :", y_train.shape)
print("Testing Labels :", y_test.shape)

Training Features : (92, 10)
Testing Features : (23, 10)
Training Labels : (92,)
Testing Labels : (23,)

#Decision tree
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score, confusion_matrix,
classification_report
dt_model = DecisionTreeClassifier(random_state=42)
dt_model.fit(X_train, y_train)
dt_pred = dt_model.predict(X_test)
dt_accuracy = accuracy_score(y_test, dt_pred)
print("Decision Tree Accuracy :", dt_accuracy)
cm = confusion_matrix(y_test, dt_pred)
plt.figure(figsize=(5,4))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Decision Tree Confusion Matrix")
plt.show()
print(classification_report(y_test, dt_pred))

Decision Tree Accuracy : 1.0
```



	precision	recall	f1-score	support
0	1.00	1.00	1.00	20
1	1.00	1.00	1.00	3
accuracy			1.00	23
macro avg	1.00	1.00	1.00	23
weighted avg	1.00	1.00	1.00	23

#Random forest

```
from sklearn.ensemble import RandomForestClassifier
rf_model = RandomForestClassifier(
    n_estimators=100,
    random_state=42
)
rf_model.fit(X_train, y_train)
rf_pred = rf_model.predict(X_test)
rf_accuracy = accuracy_score(y_test, rf_pred)
print("Random Forest Accuracy :", rf_accuracy)
print(classification_report(y_test, rf_pred))
```

Random Forest Accuracy : 1.0

	precision	recall	f1-score	support
0	1.00	1.00	1.00	20
1	1.00	1.00	1.00	3

accuracy			1.00	23
macro avg	1.00	1.00	1.00	23
weighted avg	1.00	1.00	1.00	23

#KNN

```
from sklearn.neighbors import KNeighborsClassifier
knn_model = KNeighborsClassifier(n_neighbors=5)
knn_model.fit(X_train, y_train)
knn_pred = knn_model.predict(X_test)
knn_accuracy = accuracy_score(y_test, knn_pred)
print("KNN Accuracy :", knn_accuracy)
print(classification_report(y_test, knn_pred))
```

KNN Accuracy : 0.9130434782608695

	precision	recall	f1-score	support
0	0.91	1.00	0.95	20
1	1.00	0.33	0.50	3
accuracy			0.91	23
macro avg	0.95	0.67	0.73	23
weighted avg	0.92	0.91	0.89	23

#XGBoost

```
!pip install xgboost
from xgboost import XGBClassifier
xgb_model = XGBClassifier(
    random_state=42,
    eval_metric='logloss'
)

xgb_model.fit(X_train, y_train)

xgb_pred = xgb_model.predict(X_test)
xgb_accuracy = accuracy_score(y_test, xgb_pred)

print("XGBoost Accuracy :", xgb_accuracy)
print(classification_report(y_test, xgb_pred))
```

Requirement already satisfied: xgboost in
 /usr/local/lib/python3.12/dist-packages (3.3.0)
 Requirement already satisfied: numpy in
 /usr/local/lib/python3.12/dist-packages (from xgboost) (2.0.2)
 Requirement already satisfied: nvidia-nccl-cu12 in
 /usr/local/lib/python3.12/dist-packages (from xgboost) (2.30.7)
 Requirement already satisfied: scipy in
 /usr/local/lib/python3.12/dist-packages (from xgboost) (1.16.3)
 XGBoost Accuracy : 1.0

	precision	recall	f1-score	support
0	1.00	1.00	1.00	20
1	1.00	1.00	1.00	3
accuracy			1.00	23
macro avg	1.00	1.00	1.00	23
weighted avg	1.00	1.00	1.00	23

#Comparison

```
results = pd.DataFrame({
```

```
    "Model":[
        "Decision Tree",
        "Random Forest",
        "KNN",
        "XGBoost"
    ],
```

```
    "Accuracy":[
        dt_accuracy,
        rf_accuracy,
        knn_accuracy,
        xgb_accuracy
    ]

```

```
})
```

```
results
```

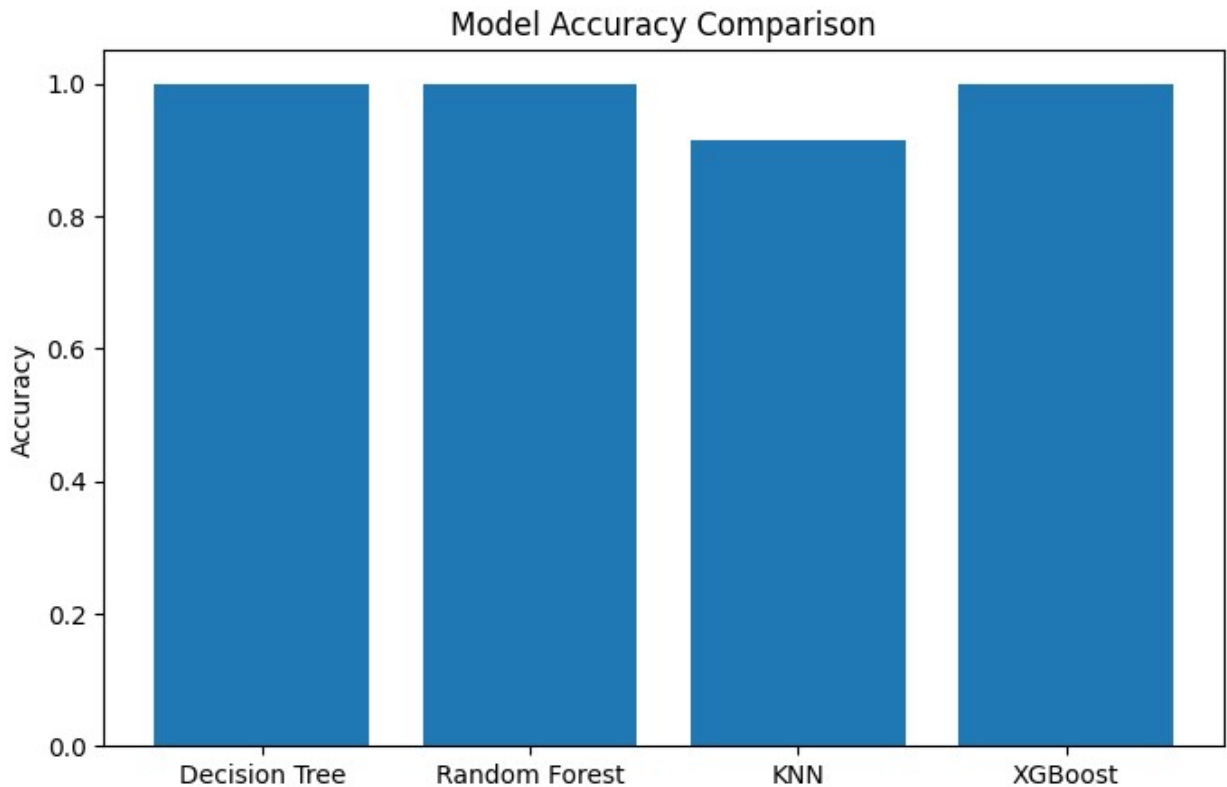
```
{
  "summary": {
    "name": "results",
    "rows": 4,
    "fields": [
      {
        "column": "Model",
        "properties": {
          "dtype": "string",
          "num_unique_values": 4,
          "samples": [
            "Random Forest",
            "XGBoost",
            "Decision Tree"
          ],
          "semantic_type": "",
          "description": ""
        }
      },
      {
        "column": "Accuracy",
        "properties": {
          "dtype": "number",
          "std": 0.043478260869565244,
          "min": 0.9130434782608695,
          "max": 1.0,
          "num_unique_values": 2,
          "samples": [
            0.9130434782608695,
            1.0
          ],
          "semantic_type": "",
          "description": ""
        }
      }
    ]
  },
  "type": "dataframe",
  "variable_name": "results"
}
```

#Accuracy bar chart

```
plt.figure(figsize=(8,5))
```

```
plt.bar(results["Model"],
        results["Accuracy"])
```

```
plt.title("Model Accuracy Comparison")
plt.ylabel("Accuracy")
plt.show()
```



```
results.sort_values(by="Accuracy", ascending=False)

{"summary": "{\n  \"name\": \"results\",\n  \"rows\": 4,\n  \"fields\": [\n    {\n      \"column\": \"Model\",\n      \"properties\": {\n        \"dtype\": \"string\",\n        \"num_unique_values\": 4,\n        \"samples\": [\n          \"Random Forest\",\n          \"KNN\",\n          \"Decision Tree\",\n          \"Random Forest\"\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      }\n    },\n    {\n      \"column\": \"Accuracy\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 0.043478260869565244,\n        \"min\": 0.9130434782608695,\n        \"max\": 1.0,\n        \"num_unique_values\": 2,\n        \"samples\": [\n          0.9130434782608695,\n          1.0\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      }\n    }\n  ]\n}", "type": "dataframe"}

#Best model
import joblib
```



```
joblib.dump(rf_model, "flood_model.pkl")  
print("Model Saved Successfully")
```

Model Saved Successfully

```
loaded_model = joblib.load("flood_model.pkl")  
prediction = loaded_model.predict(X_test[:5])  
print(prediction)
```

```
[0 0 0 0 0]
```